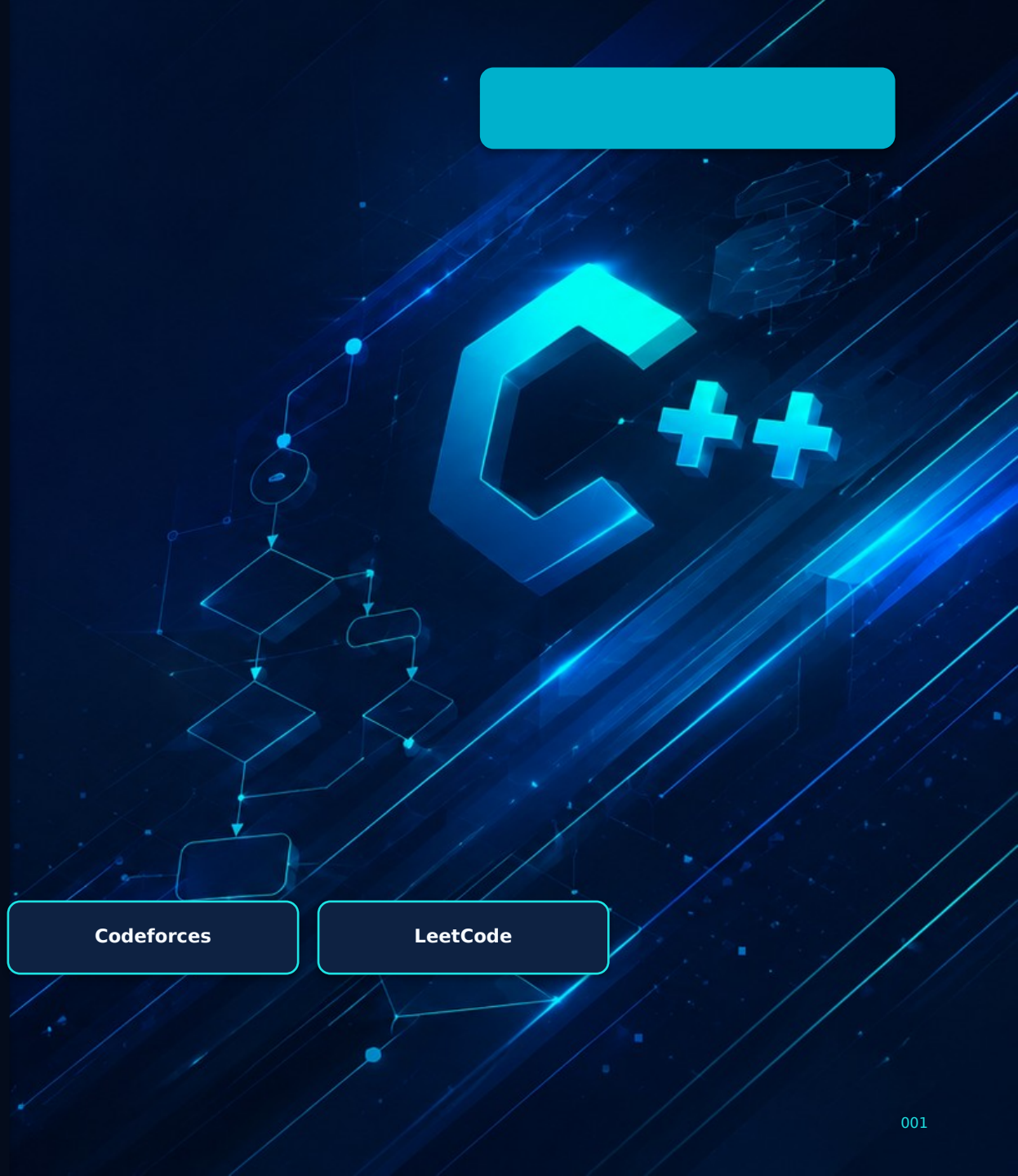
TechMaster Academy

TechMaster Academy

C++ Phase 1 Mega Recap Guide

A premium, execution-ready training file for Level 1: programming logic, C++ foundations, problem solving, submissions, sheets, streaks, contests, and mentor review.



100+ Pages | Phase 1 | C++ | Codeforces | LeetCode

This is not a normal course

The training identity

- This is an internship-style training system, not a passive lecture track.
- Students learn by reading requirements, solving problems, submitting proof, receiving feedback, and improving.
- Every page in this guide is written to prepare students before they touch the Phase 1 sheet.
- The goal is not memorizing syntax. The goal is building problem-solving behavior.
- By the end of Phase 1, every student should understand how to think, code, test, submit, and explain.

What Phase 1 must produce

- A working setup: Codeforces, LeetCode, GitHub, compiler, and training folder.
- Clean beginner C++ solutions for implementation problems.
- A daily habit around LeetCode drops and Codeforces practice.
- A clear submission workflow with links, notes, screenshots, and self-review answers.
- A first leaderboard score based on execution, not attendance.

How students should read this file

Before solving the sheet

- Read the concept first, then solve the related drills.
- Do not jump straight to Codeforces without understanding the examples.
- Every solved example includes the thinking path, not only the code.
- Write notes for mistakes. The mistake log is part of the training.
- Use this file as your reference when you get Wrong Answer, TLE, or confusion.

During the 10 days

- Day 1-3: understand the system and start guided practice.
- Day 4-6: push harder on sheet progress and LeetCode drops.
- Day 7: contest and first submission checkpoint.
- Day 8-9: review, fix, and resubmit improvements.
- Day 10: leaderboard, feedback, and readiness gate for Phase 2.

Phase 1 - 10 Day Execution Timeline

Day	Main Focus	Student Output	Mentor Output
Day 1	Phase release, setup, accounts, repo	Accounts + repo + first README	Phase guide + setup checklist
Day 2	I/O, variables, conditions	First LeetCode drop + warm-up problems	Video explanation + Q&A
Day 3	Dry run and first checkpoint	Initial progress proof	Setup review
Day 4	Loops and basic math	Second drop + loop drills	Common mistakes post
Day 5	Arrays and strings intro	Required sheet progress	Office support
Day 6	Functions and complexity intro	Third drop + checkpoint	Contest preparation
Day 7	Weekly contest	Contest score + submission form	Contest standings
Day 8	Editorial and fixing	Improved solutions	Editorial video
Day 9	Mentor review and recovery	Mistake log + fixes	Targeted feedback
Day 10	Leaderboard and gate	Final proof of Phase 1	Leaderboard + next instructions

The three tracks running together

1. Phase 1 Codeforces Sheet

- The core sheet is Codeforces-heavy and trains contest-style problem solving.
- It is split into Required, Practice, Challenge, and Bonus sections.
- Students should not wait until the last day. Sheet progress is checked every three days.
- Minimum completion is required to pass the phase gate.
- Challenge and Bonus problems build the top performer leaderboard.

2. LeetCode Streak Track

- Runs beside the Phase sheet and updates every three days.
- Each drop contains three problems: Explained, Practice, Challenge.
- The Explained problem is paired with a YouTube video or mentor walkthrough.
- This track builds interview consistency and pattern recognition.
- Streak points are separate from the Codeforces sheet score.

The weekly concept file is the command center

What the weekly PDF contains

- Full concept explanation in English.
- Solved examples with input, output, approach, code, dry run, and complexity.
- The exact Codeforces sections that should be solved this week.
- The current LeetCode drops and YouTube video links.
- Submission rules, GitHub structure, checkpoints, and rubric.

What it prevents

- Students asking: 'What should I solve now?'
- Students solving random problems outside the plan.
- Students submitting screenshots without organized proof.
- Students watching videos without executing.
- Students reaching the contest without concept preparation.

SECTION 01

Problem Solving Mindset

Before C++ syntax, students need to learn how to read, analyze, test, and explain a problem.

- Read the statement slowly.
- Extract input and output.
- Design the approach before coding.
- Test edge cases.

01

What is problem solving?

The real definition

- Problem solving is transforming a written requirement into a reliable sequence of steps.
- The language is only the implementation tool. The thinking process comes first.
- A beginner fails when they code before understanding the required behavior.
- A strong student can explain the idea before writing the first line of C++.
- Phase 1 rewards clear thinking, not fast guessing.

The output behavior

- Given inputs, produce exactly the expected output.
- Handle normal cases and edge cases.
- Respect constraints and time limits.
- Write clean enough code to debug later.
- Submit proof in the expected format.

The 6-step solving flow

Use this flow for every problem

- 1. Read the story and remove unnecessary words.
- 2. Identify the input values and their types.
- 3. Identify the output format exactly.
- 4. Write the approach in plain English.
- 5. Convert the approach into C++ code.
- 6. Test with sample cases and edge cases.

Why this matters

- It reduces Wrong Answer caused by misunderstanding.
- It protects you from coding the wrong problem.
- It makes explanations easier for mentor review.
- It trains interview communication from Phase 1.
- It builds a repeatable habit for harder phases.

Reading problem statements

What to underline mentally

- Input count: one value, many values, or multiple test cases.
- Data type clues: integer, decimal, character, string.
- Required output: one answer, many lines, formatted text.
- Constraints: how large can input get?
- Special rules: rounding, inclusive ranges, impossible cases.

Common beginner mistakes

- Ignoring the number of test cases.
- Printing extra spaces or prompts.
- Assuming input order instead of reading it.
- Missing equality cases such as <= or >=.
- Solving the sample only, not the general rule.

Input and output precision

Competitive programming is strict

- Do not print messages like 'Enter number'.
- Print exactly what the problem asks.
- A single extra word can cause Wrong Answer.
- When output requires multiple lines, match the line breaks.
- For decimals, check if fixed precision is required.

Phase 1 rule

- Your code must be judge-ready.
- No decoration in output.
- No debugging prints in submitted code.
- No extra prompts for the user.
- Always test the output visually before submitting.

Dry run

Test before the judge tests you

- Dry run means executing your algorithm manually on paper or in your head.
- Track variable values step by step.
- Use small inputs first, then edge cases.
- Dry run catches logic errors before submission.
- Every student must dry run at least one submitted solution in notes.

Recommended table

- Step number
- Current input/value
- Important variables
- Condition result
- Output so far

Edge cases

The cases that break weak solutions

- Smallest input value.
- Largest input value.
- All values equal.
- Already sorted or reverse sorted data.
- Empty-like cases when allowed.
- Negative numbers when constraints allow them.

How to write edge notes

- List 3 tests after every serious problem.
- Explain what each test is trying to catch.
- Do not trust only the sample test.
- If a solution fails, add the failed case to your mistake log.
- Edge case thinking is part of the score.

Accepted vs Wrong Answer

Judge feedback is a signal

- Accepted means the solution passed all hidden tests.
- Wrong Answer means the output does not match for at least one hidden case.
- Compilation Error means the code did not build.
- Time Limit Exceeded means your approach is too slow or stuck.
- Runtime Error often means invalid memory access or division by zero.

What to do after failure

- Do not spam submissions without thinking.
- Re-read the statement.
- Create your own tests.
- Check boundaries and data types.
- Write one sentence explaining the suspected bug.

Approach notes

Write the idea before the code

- A good note says what the solution does, not what every line does.
- Example: 'We loop through all numbers and count the values divisible by 2'.
- Keep it short but precise.
- Mention complexity when possible.
- These notes later become interview answers.

Bad vs good

- Bad: 'I used for loop'.
- Good: 'I scan the array once and keep the maximum value'.
- Bad: 'I used if'.
- Good: 'I check if the number is inside the allowed range'.
- Good notes prove understanding.

Debugging behavior

Debug like an engineer

- Reproduce the bug with a small input.
- Print important variables while debugging only.
- Remove debug prints before submission.
- Compare expected output with actual output.
- Ask for help with statement link, code, error, and what you tried.

Group question format

- Problem link
- Your code or GitHub link
- Input that fails
- Expected output
- Actual output
- What you already checked

Explanation skill

The student must be able to speak

- Explain the problem in your own words.
- Explain the main idea without reading code.
- Explain why it works.
- Explain complexity simply.
- Mention one edge case you handled.
- This skill appears in mentor review and mock interview.

One-minute template

- The input gives us...
- We need to output...
- My idea is...
- I loop/check/calculate...
- The complexity is...
- One edge case is...

SECTION 02

C++ Foundations

Students now move from thinking into clean beginner C++ syntax that works on online judges.

- Program structure
- Variables and types
- Input and output
- Operators

02

Minimal judge-ready C++ program

This is the base shape of most Phase 1 solutions. Students should understand every line before solving the sheet.

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    // write your solution here
    return 0;
}
```

What to notice

- #include brings the needed library tools.
- main is the starting point of the program.
- Fast I/O lines help with online judges.
- return 0 means the program ended successfully.
- Do not print prompts unless the problem asks.

Variables

Think of variables as named boxes

- A variable stores a value that your program can use and update.
- Choose meaningful names: age, sum, count, maxValue.
- Avoid names like a, b, c unless the problem is tiny.
- Initialize variables before using them.
- The variable type decides what kind of value can be stored.

Most used Phase 1 types

- int: normal integer values.
- long long: large integer values.
- double: decimal numbers.
- char: one character.
- string: text.
- bool: true or false.

Data type selection

Wrong type can break correct logic

- Use int for small counts and indexes.
- Use long long when multiplication or large constraints appear.
- Use double when the output requires decimal values.
- Use char for one letter and string for words or full text.
- Do not use float unless you have a specific reason.

Phase 1 safety rule

- When unsure about integer size, use long long for calculations.
- Array indexes are usually int.
- Count variables are usually int or long long depending on constraints.
- Do not mix strings and numbers without clear conversion.
- Read constraints before choosing types.

Input with cin

cin reads values in order

- cin reads from standard input exactly as the judge provides values.
- Spaces and new lines are usually treated as separators for numbers and words.
- For strings with spaces, getline is needed, but Phase 1 mostly uses simple strings.
- If the problem gives n then n values, read n first, then loop n times.
- Never assume values without reading them.

Common mistakes

- Reading variables in the wrong order.
- Forgetting to read n before the loop.
- Using char when string is required.
- Expecting cin to read spaces inside a sentence.
- Writing prompts like 'Enter x'.

Output with cout

cout must match exactly

- Use cout to print the final answer.
- Use '\n' for a new line.
- Avoid endl unless you need flushing; '\n' is faster.
- Print spaces carefully between values.
- Remove debug output before submission.

Output patterns

- Single value: cout << ans << '\n';
- Two values: cout << a << ' ' << b << '\n';
- Text result: cout << "YES\n";
- Multiple lines: print inside a loop.
- Formatted decimals: use fixed << setprecision(k).

Arithmetic operators

Build formulas carefully

- + adds, - subtracts, * multiplies, / divides, % gives remainder.
- Integer division removes the decimal part.
- Modulo works with integers and is heavily used for parity and digits.
- Use parentheses to make formulas clear.
- Beware of overflow when multiplying large values.

Examples

- n % 2 == 0 checks even number.
- lastDigit = n % 10 gets the last digit.
- n = n / 10 removes the last digit.
- average = sum / count may truncate if both are integers.
- area = width * height can overflow int if values are large.

Comparison operators

Conditions need true or false

- == checks equality.
- != checks not equal.
- > and < compare strict order.
- >= and <= include equality.
- A comparison returns a boolean result.

Mistakes to avoid

- Using = instead of == inside a condition.
- Forgetting equality in boundary cases.
- Writing chained comparisons like 1 <= x <= 10 in C++.
- Comparing decimals carelessly.
- Comparing strings without understanding lexicographic order.

Logical operators

Combine conditions

- && means AND: both sides must be true.
- || means OR: at least one side must be true.
- ! means NOT: reverses the condition.
- Use parentheses for readability.
- Complex logic should be tested with edge cases.

Example decisions

- x >= 1 && x <= 10 means x is inside range.
- grade == 'A' || grade == 'B' means either A or B.
- !(x > 0) means x is not positive.
- year % 4 == 0 can be part of leap-year logic.
- Use truth tables for confusing cases.

Assignment shortcuts

Write updates clearly

- x = x + 1 can be written as x++.
- sum += value means sum = sum + value.
- count++ is common when counting matches.
- product *= value is used for multiplication accumulation.
- Use shortcuts only when they make code clearer.

Phase 1 habit

- Start with readable updates.
- Do not overuse clever syntax.
- Initialize accumulators before loops.
- Check if increment happens at the correct place.
- Wrong update position causes wrong answers.

Comments

Use comments for thinking, not noise

- Comments can explain the idea or a tricky condition.
- Do not comment every obvious line.
- A comment should help future you understand the approach.
- Remove temporary debug comments if they confuse the solution.
- README notes are more important than excessive in-code comments.

Good comments

- // count numbers divisible by k
- // try all possible values from 1 to n
- // handle the case where no value was found
- // update maximum after reading each number
- // skip invalid characters

Compilation errors

The first wall for beginners

- Missing semicolon after a statement.
- Using a variable before declaring it.
- Mismatched braces { }.
- Wrong capitalization: cout is not Cout.
- Forgetting quotes around strings.
- Using single quotes for multi-character text.

Fixing process

- Read the first error, not only the last one.
- Check the line before the reported line.
- Format your code to see braces clearly.
- Compile locally before submitting when possible.
- Keep one working solution template.

Input/output example

This example reads two integers, calculates their sum, and prints only the answer. No prompts. No extra words.

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int a, b;
    cin >> a >> b;

    int sum = a + b;
    cout << sum << '\n';
    return 0;
}
```

What to notice

- If input is 3 5, output is 8.
- The judge already knows the input format.
- You should not write 'Enter a and b'.
- The variable sum stores the result before printing.
- For large values, use long long instead of int.

SECTION 03

Conditions

Conditions let the program choose between paths. Phase 1 students must master boundary logic and equality cases.

- if / else
- nested decisions
- logical operators
- range checks

03

if statement

One condition, one decision

- Use if when something should happen only when a condition is true.
- The condition must produce true or false.
- The block inside braces runs only when the condition is true.
- Braces protect you from accidental logic errors.
- A simple if does nothing when the condition is false.

Template

- if (condition) {
- // code if true
- }
- Use this for optional actions.
- Example: if number is even, print EVEN.

if / else

Two possible paths

- Use if/else when exactly one of two blocks should run.
- The else block runs when the if condition is false.
- This is ideal for even/odd, pass/fail, positive/non-positive.
- Keep conditions simple when possible.
- Test both branches.

Common examples

- Even or odd
- Positive or negative
- Accepted or rejected
- Inside range or outside range
- Can buy or cannot buy

else if chain

More than two paths

- Use else if when there are multiple categories.
- The program checks conditions from top to bottom.
- The first true branch runs; the rest are skipped.
- Order matters when ranges overlap.
- Finish with else for the remaining case when needed.

Examples

- Grade classification
- Age group
- Number sign: negative, zero, positive
- Triangle type
- Ticket pricing by category

Nested conditions

A decision inside another decision

- Nested if can express dependent conditions.
- Do not nest too deeply if a simpler logical expression works.
- Indentation is important for readability.
- Nested logic is common in validation problems.
- Mentor review checks clarity, not only correctness.

When to use

- When second check only matters if first check passes.
- When categories have subcategories.
- When a problem has priority rules.
- When multiple independent if statements would be wrong.
- When the story says 'if this, then check that'.

Range checks

C++ does not support mathematical chaining

- Do not write 1 <= x <= 10 in C++.
- Write x >= 1 && x <= 10.
- Use AND when both boundaries must be true.
- Use OR when one of multiple cases is enough.
- Always test boundary values: 1 and 10 in this example.

Boundary test set

- x = 0 should be outside.
- x = 1 should be inside.
- x = 5 should be inside.
- x = 10 should be inside.
- x = 11 should be outside.

Solved example - maximum of three numbers

Problem

Given three integers a, b, and c, print the maximum value.

Approach

- Read a, b, c.
- Assume maxValue = a.
- If b is greater than maxValue, update it.
- If c is greater than maxValue, update it.
- Print maxValue.

```
int a, b, c;
cin >> a >> b >> c;

int mx = a;
if (b > mx) mx = b;
if (c > mx) mx = c;

cout << mx << '\n';
```

Edge checks all equal | negative values | largest first | largest last

Solved example - grading system

Problem

Given a score from 0 to 100, print A, B, C, D, or F using ranges.

Approach

- Read score.
- Check highest range first.
- Use else if to prevent multiple outputs.
- Include equality boundaries.
- Print exactly one grade.

```
int score;
cin >> score;

if (score >= 90) cout << "A\n";
else if (score >= 80) cout << "B\n";
else if (score >= 70) cout << "C\n";
else if (score >= 60) cout << "D\n";
else cout << "F\n";
```

Edge checks 90 | 89 | 60 | 59

Solved example - inside rectangle

Problem

Given x and y coordinates, decide whether the point is inside a rectangle from (0,0) to (w,h).

Approach

- Read x, y, w, h.
- Inside means x is between 0 and w and y is between 0 and h.
- Both x condition and y condition must be true.
- Use AND between all boundaries.
- Decide whether boundary is considered inside based on statement.

```
int x, y, w, h;
cin >> x >> y >> w >> h;

if (x >= 0 && x <= w && y >= 0 && y <= h)
    cout << "INSIDE\n";
else
    cout << "OUTSIDE\n";
```

Edge checks on edge | negative x | x>w | y>h

Conditions Drill Pack

Drill 1

Even or Odd

Read an integer and print EVEN if divisible by 2, otherwise ODD.

Drill 2

Maximum of Two

Read two integers and print the larger one. If equal, print EQUAL.

Drill 3

Temperature Status

Read temperature and print COLD, NORMAL, or HOT based on defined ranges.

Drill 4

Login Validator

Read username length and password length, then decide if account is valid.

Drill 5

Triangle Validity

Read three sides and check if they can form a triangle.

Drill 6

Discount Rule

Read price and membership flag, then calculate final price based on rules.

SECTION 04

Loops

Loops are the engine of Phase 1. Students use loops to repeat reading, counting, searching, summing, and simulating.

- for loops
- while loops
- nested loops
- counters and accumulators

04

Why loops matter

Most problems repeat an action

- Read n numbers.
- Count how many values match a condition.
- Calculate the sum or average.
- Search for a maximum or minimum.
- Repeat until a value becomes zero.
- Generate a pattern or simulate steps.

Loop thinking

- What changes every iteration?
- When should the loop stop?
- What variable should be updated?
- What is the initial value?
- What must be printed after the loop?

for loop

Best when the number of iterations is known

- Use for when you know how many times to repeat.
- Classic shape: for (int i = 0; i < n; i++).
- The loop variable usually starts at 0 for arrays.
- For counting from 1 to n, use i = 1; i <= n.
- Be careful with < vs <=.

Common uses

- Read n values.
- Print numbers from 1 to n.
- Traverse an array.
- Count matching elements.
- Repeat fixed number of test cases.

while loop

Best when the stop condition is dynamic

- Use while when you repeat until a condition becomes false.
- The condition is checked before each iteration.
- Make sure something changes inside the loop.
- while is common in digit problems.
- while is also used for input until sentinel value.

Common uses

- Process digits while n > 0.
- Repeat until user gives zero.
- Simulate until a value reaches target.
- Search while boundaries are valid.
- Keep reading while input exists.

Counters

Counting matches

- A counter starts at 0.
- Increase it when a condition is true.
- Print the counter after the loop.
- Counters answer 'how many'.
- Always verify the condition that triggers counting.

Examples

- Count even numbers.
- Count positive values.
- Count vowels in a string.
- Count numbers greater than x.
- Count failed students.

Accumulators

Building a total

- An accumulator stores a running result.
- sum starts at 0.
- product often starts at 1.
- maximum can start as first value or a very small value.
- Update inside the loop after reading each value.

Examples

- Sum of n numbers.
- Product of digits.
- Total price.
- Number of steps.
- Score accumulation.

Nested loops

A loop inside another loop

- Nested loops are used for grids, patterns, and pair checking.
- If outer loop runs n times and inner loop runs m times, total work is n*m.
- Nested loops can become too slow for large input.
- Use clear variable names: i for rows, j for columns.
- Print new line after each row in pattern problems.

Phase 1 warning

- Nested loops are fine for small constraints.
- Do not use nested loops blindly.
- Estimate operations before submitting.
- If n = 100000, O(n^2) is usually impossible.
- Complexity starts here.

break and continue

Control loop flow carefully

- break exits the current loop immediately.
- continue skips the rest of the current iteration.
- Use break when the answer is already found.
- Use continue to ignore invalid cases.
- Do not overuse them when simple conditions are clearer.

Examples

- Stop when first negative number appears.
- Skip values equal to zero.
- Stop search after finding target.
- Ignore characters that are not letters.
- Leave a menu when user chooses exit.

Infinite loops

The classic beginner trap

- A loop becomes infinite when its stop condition never changes.
- while loops are especially risky.
- Always update the variable that controls the condition.
- Test with small inputs.
- If a program hangs, check the loop condition first.

Debug checklist

- Is the loop variable initialized?
- Is it updated?
- Can the condition ever become false?
- Is continue skipping the update?
- Are you using == instead of = or vice versa?

Counting even numbers

This code reads n integers and counts how many of them are even. It demonstrates input loop, counter, and condition inside a loop.

```
int n;
cin >> n;

int evenCount = 0;
for (int i = 0; i < n; i++) {
    int x;
    cin >> x;
    if (x % 2 == 0) {
        evenCount++;
    }
}

cout << evenCount << '\n';
```

What to notice

- Read n first.
- Loop exactly n times.
- Read one value per iteration.
- Modulo checks divisibility by 2.
- Counter increases only when condition is true.

Solved example - sum from 1 to n

Problem

Given n, print the sum of all integers from 1 to n using a loop.

Approach

- Start sum at 0.
- Loop i from 1 to n.
- Add i to sum each iteration.
- Print sum after the loop.
- Use long long for safety when n can be large.

```
long long n;
cin >> n;

long long sum = 0;
for (long long i = 1; i <= n; i++) {
    sum += i;
}

cout << sum << '\n';
```

Edge checks n=1 | n=0 if allowed | large n

Solved example - count digits

Problem

Given a positive integer n, print how many digits it has.

Approach

- Use while n > 0.
- Each iteration removes last digit using n /= 10.
- Increase count each time.
- Special case: n = 0 has one digit.
- This pattern appears in many math problems.

```
long long n;
cin >> n;

if (n == 0) {
    cout << 1 << '\n';
} else {
    int cnt = 0;
    while (n > 0) {
        cnt++;
        n /= 10;
    }
    cout << cnt << '\n';
}
```

Edge checks 0 | 7 | 12345

Loop Drill Pack

Drill 1

Print 1 to N

Read n and print all numbers from 1 to n, one per line.

Drill 2

Sum of Evens

Read n numbers and print the sum of the even values only.

Drill 3

Count Positives

Read n numbers and count how many are greater than zero.

Drill 4

Digit Sum

Read n and calculate the sum of its digits using while.

Drill 5

First Divisible

Read n numbers and print the first number divisible by k.

Drill 6

Pattern Rectangle

Read rows and columns and print a rectangle of stars.

SECTION 05

Math Basics

Phase 1 math is not advanced number theory. It is the basic toolkit needed for implementation problems.

- Modulo
- Integer division
- ceil division
- digits

05

Modulo

The remainder operator

- a % b gives the remainder after dividing a by b.
- Use modulo to check divisibility.
- n % 2 tells you if a number is even or odd.
- n % 10 gives the last digit.
- Modulo is only for integer types in C++.

Examples

- 10 % 3 = 1
- 15 % 5 = 0
- 123 % 10 = 3
- n % k == 0 means n is divisible by k
- Digit problems use modulo constantly.

Integer division

C++ removes the decimal part

- 5 / 2 is 2 when both operands are integers.
- If you need decimal output, use double.
- Integer division is useful for removing digits.
- n / 10 removes the last digit.
- Be careful in average calculations.

Examples

- int avg = sum / n may lose decimals.
- double avg = 1.0 * sum / n keeps decimals.
- 123 / 10 = 12
- 9 / 3 = 3
- 1 / 2 = 0 as integer division.

Ceil division

Rounding up integer division

- Sometimes you need the number of groups/pages/days.
- For positive integers, ceil(a / b) can be written as (a + b - 1) / b.
- Use long long if a and b can be large.
- This appears in packaging and capacity problems.
- Always check if the problem wants floor or ceil.

Example

- 10 items, box size 3 -> 4 boxes
- Formula: (10 + 3 - 1) / 3 = 4
- 9 items, box size 3 -> 3 boxes
- 1 item, box size 3 -> 1 box
- 0 items depends on statement.

Min and max

Keeping best values

- Maximum tracking starts with the first value or very small value.
- Minimum tracking starts with the first value or very large value.
- Update while scanning values.
- C++ has min() and max() built-ins.
- For arrays, scan once and update.

Beginner rule

- If input has at least one number, initialize from the first number.
- Do not initialize max to 0 if numbers can be negative.
- Do not forget to read all remaining numbers.
- Print after loop, not inside unless required.
- Use meaningful names: maxValue, minValue.

Overflow

Correct logic can fail with wrong type

- int has limited range.
- Multiplication grows fast.
- Use long long for products and large sums.
- If a, b are int, a*b happens as int unless casted or stored carefully.
- Hidden tests often include large boundaries.

Safe habits

- long long area = 1LL * width * height;
- Use long long for sum of many numbers.
- Read constraints before coding.
- Use 1LL to force long long multiplication.
- Do not ignore overflow in math problems.

Digits pattern

Many math problems use the same loop

- Last digit: n % 10.
- Remove last digit: n /= 10.
- Repeat while n > 0.
- Use this for digit sum, digit count, reverse number, product of digits.
- Handle n = 0 separately when needed.

Mental model

- n = 123
- last = 3, n becomes 12
- last = 2, n becomes 1
- last = 1, n becomes 0
- Loop ends.

Solved example - reverse a number

Problem

Given a non-negative integer n, print its digits reversed as a number.

Approach

- Start reversed = 0.
- Take last digit using n % 10.
- Add digit to reversed: reversed = reversed * 10 + digit.
- Remove last digit using n /= 10.
- Special case for n = 0.

```
long long n;
cin >> n;

long long rev = 0;
if (n == 0) {
    cout << 0 << '\n';
} else {
    while (n > 0) {
        int digit = n % 10;
        rev = rev * 10 + digit;
        n /= 10;
    }
    cout << rev << '\n';
}
```

Edge checks 0 | 1200 | 1001

Math Basics Drill Pack

Drill 1

Last Digit

Read n and print its last digit.

Drill 2

Divisibility

Read n and k, print YES if n is divisible by k.

Drill 3

Required Buses

Read students and bus capacity, print number of buses needed.

Drill 4

Digit Product

Read n and print product of its digits.

Drill 5

Maximum Product

Read two dimensions and calculate area safely using long long.

Drill 6

Clock Conversion

Read total minutes and print hours and remaining minutes.

SECTION 06

Arrays

Arrays let students store many values and process them with loops. This is the bridge from single-value problems to real problem solving.

- Storage
- Traversal
- sum/min/max
- counting

06

What is an array?

Many values under one name

- An array stores multiple values of the same type.
- Values are accessed by index.
- C++ indexes start from 0.
- If size is n, valid indexes are 0 to n-1.
- Arrays are used when you need to remember values for later.

When to use

- You need to scan values more than once.
- You need to compare elements with neighbors.
- You need to sort values later.
- You need frequency or positions.
- You need to output processed values after reading all input.

Reading arrays

The classic input loop

- Read n first.
- Create an array or vector of size n.
- Loop i from 0 to n-1.
- Read each value into a[i].
- Do not access a[n]; it is outside the array.

Phase 1 rule

- Prefer vector<int> a(n) for safety and dynamic size.
- Use long long values if constraints are large.
- Keep index variables clear.
- Test n = 1.
- Never assume array is sorted unless statement says so.

Array traversal

Visit each element once

- Traversal means looping over all elements.
- Most Phase 1 array problems are traversal with a condition or update.
- Keep answer variables outside the loop.
- Update answer inside the loop.
- Print after traversal unless problem asks each step.

Examples

- Sum all elements.
- Find maximum.
- Count negative numbers.
- Check if all values are positive.
- Print elements in reverse.

Maximum and minimum

Tracking best value

- Initialize maxValue with a[0].
- Loop from i = 1 to n-1.
- If a[i] > maxValue, update it.
- Same idea for minimum with <.
- This works with negative numbers too.

Common wrong version

- Initializing maxValue = 0 fails when all numbers are negative.
- Printing inside the loop prints intermediate values.
- Forgetting to read all elements.
- Using wrong comparison sign.
- Accessing a[0] when n can be 0.

Counting in arrays

Condition over many values

- Start count at 0.
- Loop over elements.
- If the element satisfies the condition, increment count.
- Examples: even, positive, greater than x, divisible by k.
- Counting is one of the most common Phase 1 tasks.

Explanation sentence

- I scan the array once and increase a counter whenever the current element satisfies the required condition. This is O(n).

Frequency basics

How many times each value appears

- If values are small, you can use a frequency array.
- freq[x]++ counts occurrence of value x.
- Frequency is useful for repeated values and counting unique values later.
- Make sure value x is inside freq array range.
- Phase 2 expands this with maps.

Safety

- Check constraints before using freq array.
- If values can be negative or huge, wait for map in Phase 2.
- Initialize frequency array with zeros.
- Use vector<int> freq(maxValue + 1).
- Do not go out of range.

Out of bounds

The silent killer

- Valid indexes for size n are 0 to n-1.
- a[n] is invalid.
- a[-1] is invalid.
- Neighbor checks need boundaries.
- Runtime Error often comes from invalid index.

Boundary examples

- If checking a[i+1], ensure i+1 < n.
- If checking a[i-1], ensure i > 0.
- For reverse loop: for (int i=n-1; i>=0; i--).
- For n=1, neighbor problems need special care.
- Always test smallest n.

Arrays and complexity

Traversal is usually O(n)

- One loop over n elements is O(n).
- Nested loops over array pairs are O(n^2).
- O(n^2) may be fine for n <= 1000 but not n = 100000.
- Do not sort in Phase 1 unless taught or allowed.
- If you only need max/sum/count, one pass is enough.

Mentor question

- Can your solution work if n = 100000?
- If not, why?
- What operation is repeated most?
- Is there a nested loop?
- Can one pass solve it?

Read and print array in reverse

This example reads n values, stores them, then prints them in reverse order. We need storage because output order is different from input order.

```
int n;
cin >> n;
vector<int> a(n);

for (int i = 0; i < n; i++) {
    cin >> a[i];
}

for (int i = n - 1; i >= 0; i--) {
    cout << a[i] << ' ';
}
cout << '\n';
```

What to notice

- vector<int> a(n) stores n integers.
- First loop reads values.
- Second loop starts from n-1 and moves down to 0.
- i >= 0 works because i is int.
- Be careful with unsigned indexes in reverse loops.

Solved example - maximum element

Problem

Given n and an array of n integers, print the maximum element.

Approach

- Read n and the array.
- Initialize mx with the first element.
- Scan remaining elements.
- Update mx if current value is larger.
- Print mx after the loop.

```
int n;
cin >> n;
vector<int> a(n);
for (int i = 0; i < n; i++) cin >> a[i];

int mx = a[0];
for (int i = 1; i < n; i++) {
    if (a[i] > mx) mx = a[i];
}
cout << mx << '\n';
```

Edge checks n=1 | all negative | all equal

Solved example - count greater than x

Problem

Given n, x, and n numbers, count how many numbers are greater than x.

Approach

- Read n and x.
- Start count at 0.
- Loop n times and read each value.
- If value > x, increment count.
- We do not need to store the array because one pass is enough.

```
int n, x;
cin >> n >> x;

int cnt = 0;
for (int i = 0; i < n; i++) {
    int value;
    cin >> value;
    if (value > x) cnt++;
}

cout << cnt << '\n';
```

Edge checks none greater | all greater | equal to x

Array Drill Pack

Drill 1

Array Sum

Read n and n integers, then print their sum.

Drill 2

Minimum Element

Read an array and print its minimum value.

Drill 3

Count Divisible

Read array and k, count values divisible by k.

Drill 4

Reverse Output

Print the array elements in reverse input order.

Drill 5

First Occurrence

Read array and x, print first index where x appears or -1.

Drill 6

Frequency Small

Values are between 0 and 100. Count occurrences of each value.

SECTION 07

Strings

Strings turn character-by-character thinking into code. Students must handle length, traversal, counting, and simple transformations.

- string input
- length
- characters
- palindrome

07

string vs char

Text vs one character

- char stores one character like 'a'.
- string stores a sequence of characters like "hello".
- Use single quotes for char and double quotes for string.
- String indexes start from 0 like arrays.
- s.size() returns the number of characters.

Examples

- char c = 'A';
- string s = "TechMaster";
- s[0] is 'T'.
- s[s.size()-1] is the last character.
- Do not use char for full words.

Reading strings

cin reads one token

- cin >> s reads until the first space.
- For Phase 1, most string problems use one word.
- If the statement gives full line text, getline is needed.
- Be careful after reading numbers before getline.
- Output should match the required format exactly.

Common inputs

- One word
- A binary string
- A lowercase string
- A string of digits
- A code or password without spaces.

String traversal

Loop over characters

- Use for (int i = 0; i < s.size(); i++).
- s[i] is the current character.
- You can count, compare, or transform characters.
- Avoid accessing s[s.size()].
- Test strings of length 1.

Common tasks

- Count vowels.
- Count uppercase letters.
- Check if character appears.
- Print characters in reverse.
- Calculate digit sum of a numeric string.

Palindrome

Same forward and backward

- A string is palindrome if s[i] equals s[n-1-i] for all valid i.
- Only need to check half the string.
- Use a boolean flag to track validity.
- Stop early if mismatch is found.
- Case sensitivity depends on statement.

Examples

- madam is palindrome.
- racecar is palindrome.
- hello is not.
- a is palindrome.
- aa is palindrome.

Character checks

Use comparisons or cctype functions

- You can compare characters using ==, <, >.
- Digits are between '0' and '9'.
- Lowercase letters are between 'a' and 'z'.
- Uppercase letters are between 'A' and 'Z'.
- cctype functions exist, but understand the logic first.

Examples

- if (c >= '0' && c <= '9')
- if (c == 'a' || c == 'e')
- if (s[i] == target)
- digitValue = c - '0'
- lowercase index = c - 'a'

String frequency

Count letters

- For lowercase English letters, use vector<int> freq(26).
- Index is c - 'a'.
- Increase freq[index] for each character.
- This is useful for repeated letters and anagram basics.
- Make sure characters are lowercase before using this.

Safety

- If input has uppercase, convert or handle separately.
- If input has digits, use freq(10) with c - '0'.
- Never index freq with raw char value unless intended.
- Check the statement's character set.
- Print only what the problem asks.

Count vowels in a string

This example reads one word and counts how many lowercase vowels it contains.

```
string s;
cin >> s;

int cnt = 0;
for (int i = 0; i < (int)s.size(); i++) {
    char c = s[i];
    if (c == 'a' || c == 'e' || c == 'i' ||
        c == 'o' || c == 'u') {
        cnt++;
    }
}

cout << cnt << '\n';
```

What to notice

- s.size() gives length.
- Each character is checked once.
- The condition uses OR because any vowel is accepted.
- This is O(n), where n is string length.
- If uppercase exists, handle it separately.

Solved example - palindrome check

Problem

Given a lowercase string s, print YES if it is a palindrome, otherwise NO.

Approach

- Set ok = true.
- Compare s[i] with s[n-1-i].
- If any mismatch appears, set ok = false.
- Only loop while i < n/2.
- Print based on ok.

```
string s;
cin >> s;
int n = s.size();

bool ok = true;
for (int i = 0; i < n / 2; i++) {
    if (s[i] != s[n - 1 - i]) {
        ok = false;
        break;
    }
}

cout << (ok ? "YES" : "NO") << '\n';
```

Edge checks a | aa | ab | racecar

String Drill Pack

Drill 1

Length and Ends

Read a string and print its length, first character, and last character.

Drill 2

Count Letter

Read string s and character c, count how many times c appears.

Drill 3

Reverse String

Read a string and print it reversed.

Drill 4

Vowel Count

Count lowercase vowels in one word.

Drill 5

Digit Sum String

Read a numeric string and print the sum of its digits.

Drill 6

Palindrome

Print YES if the string is palindrome, otherwise NO.

SECTION 08

Built-ins and Functions

Students must learn when to use existing tools and how to decompose a solution into reusable logic.

- cmath
- algorithm basics
- string built-ins
- custom functions

08

Built-in functions

Do not reinvent every wheel

- C++ provides many ready functions.
- Use min and max for simple comparisons.
- Use abs for absolute value.
- Use sqrt carefully for square root.
- Use sort later in Phase 2, but understand it as a built-in.

Phase 1 examples

- max(a,b)
- min(a,b)
- abs(x)
- s.size()
- swap(a,b)
- to_string(x) when needed

Why custom functions?

Break a solution into clean parts

- A function gives a name to a repeated action.
- Functions make code easier to test and explain.
- They reduce copy/paste logic.
- They help students think in reusable blocks.
- Interviewers like clear decomposition.

Examples

- isEven(x)
- isVowel(c)
- sumDigits(n)
- maxOfThree(a,b,c)
- isPalindrome(s)

Function structure

Input, work, output

- A function may receive parameters.
- It may return a value.
- The return type tells what kind of value comes back.
- void means the function returns nothing.
- Keep functions focused on one job.

Template

- return_type function_name(parameters) {
- // work
- return value;
- }
- Call the function from main.

Parameters

Values sent into a function

- Parameters are local variables inside the function.
- They allow the same function to work with different inputs.
- Names can be different from variables in main.
- For Phase 1, pass simple values by value.
- Advanced reference parameters come later.

Example

- bool isEven(int x)
- int maxOfTwo(int a, int b)
- long long sumDigits(long long n)
- bool isVowel(char c)
- int countChar(string s, char target)

Return values

Give the answer back

- A non-void function must return a value.
- Return should match the declared type.
- Use bool functions for yes/no checks.
- Use int or long long functions for calculations.
- Do not print inside a function if the function should only calculate.

Clean habit

- Function calculates.
- main handles input/output.
- This separation makes testing easier.
- It also makes explanations cleaner.
- Avoid mixing too many responsibilities.

Function mistakes

Common errors

- Forgetting to return a value.
- Returning wrong type.
- Defining function after main without declaration in some cases.
- Using variables from main that are not visible inside the function.
- Printing inside every helper function unnecessarily.

Debugging

- Read compiler errors carefully.
- Check parameter names.
- Check return type.
- Call the function with correct arguments.
- Test function with simple values.

Function example - isEven

This function returns true if a number is even. main uses it to decide what to print.

```
bool isEven(int x) {
    return x % 2 == 0;
}

int main() {
    int n;
    cin >> n;

    if (isEven(n))
        cout << "EVEN\n";
    else
        cout << "ODD\n";

    return 0;
}
```

What to notice

- The function has one job.
- It returns bool.
- The condition in main becomes readable.
- This structure helps when logic grows.
- Do not overcomplicate tiny solutions, but learn the habit.

Solved example - sum of digits function

Problem

Write a function that returns the sum of digits of n, then use it in main.

Approach

- Create sumDigits(long long n).
- Inside function, use digit pattern.
- Return sum.
- main reads input and prints returned value.
- Handle n=0 naturally: sum remains 0.

```
long long sumDigits(long long n) {
    long long sum = 0;
    while (n > 0) {
        sum += n % 10;
        n /= 10;
    }
    return sum;
}

int main() {
    long long n;
    cin >> n;
    cout << sumDigits(n) << '\n';
}
```

Edge checks 0 | 105 | 999

Function Drill Pack

Drill 1

isPositive

Write a function that returns true if x is greater than zero.

Drill 2

maxOfThree

Write a function that returns the maximum of three numbers.

Drill 3

countDigits

Write a function that returns how many digits n has.

Drill 4

isVowel

Write a function that checks whether a character is a lowercase vowel.

Drill 5

arraySum

Write a function that returns the sum of vector elements.

Drill 6

isPalindrome

Write a function that checks if a string is palindrome.

SECTION 09

Complexity Intro

Phase 1 does not need advanced analysis, but students must know why some correct-looking solutions are too slow.

- O(1)
- O(n)
- O(n^2)
- constraints

09

What is complexity?

A simple way to estimate work

- Complexity describes how running time grows as input grows.
- We usually count repeated operations, not seconds exactly.
- The biggest repeated pattern matters most.
- Online judges enforce time limits.
- A solution can be logically correct but too slow.

Phase 1 goal

- Recognize one loop as O(n).
- Recognize nested loops as O(n^2).
- Read constraints before choosing approach.
- Avoid unnecessary repeated work.
- Explain complexity in one sentence.

O(1)

Constant work

- The program does a fixed number of operations.
- Input size does not change the main work.
- Simple formulas are usually O(1).
- Example: checking if one number is even.
- O(1) is very fast.

Examples

- a + b
- n % 2
- max(a,b)
- area = w*h
- ceil division formula

O(n)

One pass over input

- One loop from 0 to n-1 is O(n).
- Reading n values is already O(n).
- Counting, summing, max, min are usually O(n).
- O(n) is good for large n in beginner problems.
- Try to solve array scans in one pass when possible.

Examples

- Sum array
- Find maximum
- Count even numbers
- Reverse string
- Check palindrome

O(n^2)

Nested loops

- A loop inside another loop can become O(n^2).
- If n = 1000, n^2 is 1,000,000 operations.
- If n = 100000, n^2 is too large.
- Nested loops are acceptable only when constraints are small.
- Do not use pair checking without reading constraints.

Examples

- Compare every pair.
- Print n by n grid.
- Nested pattern printing.
- Brute force all pairs.
- Bubble-like repeated scans.

Constraints

The statement tells you what is possible

- Constraints are not decoration.
- They guide the expected complexity.
- Small n allows brute force.
- Large n requires one pass or smarter patterns.
- When constraints are missing, design reasonably and test.

Rough beginner guide

- n <= 100: O(n^2) may be okay.
- n <= 1000: O(n^2) often okay.
- n <= 100000: prefer O(n) or O(n log n).
- n very large: formulas may be required.
- Phase 1 mainly targets O(1), O(n), simple O(n^2).

Memory

Storage also matters

- An array of n integers uses memory proportional to n.
- If you do not need to store values, process while reading.
- Frequency arrays require a safe value range.
- Huge arrays may fail memory limits.
- Use vector with size based on constraints.

Memory decisions

- Need reverse output? Store values.
- Need only sum? No need to store.
- Need max only? No need to store.
- Need frequency? Need range knowledge.
- Need later sorting? Store values.

Complexity Cheat Sheet for Phase 1

Pattern	Typical Code	Complexity	Use When
Formula	answer = a * b	O(1)	Direct math
Single loop	for i in n	O(n)	Scan values once
Two separate loops	for n + for n	O(n)	Still linear
Nested loops	for n inside for n	O(n^2)	Small n or grids
String traversal	for char in s	O(length)	Count/check characters
Digit loop	while n > 0	O(number of digits)	Digit processing

SECTION 10

Execution, Submission, and Review

The training becomes serious when every student submits organized proof, not random screenshots.

- Codeforces sheet
- LeetCode drops
- YouTube videos
- GitHub repo

10

Codeforces Phase Sheet

The core score of Phase 1

- The sheet contains Required, Practice, Challenge, and Bonus problems.
- Required problems are mandatory for phase passing.
- Practice problems improve speed and confidence.
- Challenge problems decide stronger leaderboard positions.
- Bonus problems are for top performers and advanced students.

Student proof

- Problem links or solved count.
- GitHub folder with solution files.
- Screenshots when requested.
- Notes for selected problems.
- Self-review answers in the submission form.

LeetCode Drops

Every three days

- A drop is released on fixed days during the phase.
- Each drop includes three problems: Explained, Practice, Challenge.
- The Explained problem has a YouTube video or mentor walkthrough.
- Students submit drop progress in the same phase form or drop tracker.
- Drops build interview-style consistency beside Codeforces practice.

Drop structure

- Problem A - Explained
- Problem B - Practice
- Problem C - Challenge
- Deadline before next drop
- Points added to LeetCode streak leaderboard

YouTube videos

Video is part of the system

- Videos are not passive entertainment.
- Each video has a related action: solve, fix, or submit.
- Students should pause, dry run, and code along.
- The video problem may appear as the Explained problem in a drop.
- Video comments or notes can be used for review.

Video types

- Concept explanation
- Solved problem walkthrough
- Common mistakes review
- Contest editorial
- Submission and GitHub instructions

GitHub structure

Portfolio from day one

- Every student creates one training repo.
- Phase 1 has its own folder.
- Every week has folders for Codeforces, LeetCode drops, contest, and notes.
- Commits should happen during the phase, not only at the end.
- GitHub organization is part of the score.

Folder design

- phase-01-foundations/week-01/codeforces
- phase-01-foundations/week-01/leetcode-drops
- phase-01-foundations/week-01/contest
- phase-01-foundations/week-01/notes.md
- README.md at repo root

Submission form

What students must submit

- Name, WhatsApp number, Codeforces handle, LeetCode profile, GitHub repo link.
- Phase 1 folder link and weekly folder link.
- Solved count for required, practice, challenge, and bonus sections.
- Contest participation and score.
- Self-review answers and mistake log.

Self-review questions

- What was the hardest problem?
- What mistake did you fix?
- What concept is still unclear?
- Explain one solution approach.
- What will you improve before Phase 2?

Leaderboards

Competition with fairness

- Phase leaderboard combines sheet, drops, contest, GitHub, and explanation.
- Contest leaderboard highlights performance under pressure.
- LeetCode streak leaderboard rewards consistency.
- Most Improved leaderboard keeps weaker students motivated.
- Badges recognize different types of effort.

Badges

- Top Solver
- Best Streak
- Contest Fighter
- Best Explanation
- Clean Code Star
- Most Improved

Mentor review

Feedback is part of the product

- Mentors review common mistakes publicly.
- Selected submissions receive deeper comments.
- Students may get recovery tasks after weak submissions.
- Improvement after feedback matters.
- The goal is behavior improvement, not embarrassment.

Review focus

- Correctness
- Code clarity
- Testing
- GitHub organization
- Explanation quality
- Consistency

Phase 1 Scoring Model

Component	Points	What is checked	Proof
Codeforces Sheet	35	Required + practice progress	Sheet tracker + GitHub
LeetCode Drops	15	Drops every 3 days	Drop tracker + screenshots
Weekly Contest	25	Participation and score	Contest standing
GitHub	10	Repo structure and commits	Repo link
Explanation	10	Self-review and notes	Form answers
Bonus	5	Early submission or bonus problems	Mentor approval

Final Student Checklist Before Solving the Sheet

- ✓ I created my Codeforces account and saved my handle.
- ✓ I created my LeetCode account or confirmed my existing profile.
- ✓ I created one GitHub repo for the full training.
- ✓ I created phase-01-foundations folder.
- ✓ I understand the difference between Codeforces sheet and LeetCode drops.
- ✓ I know that LeetCode drops update every three days.
- ✓ I understand the submission form will ask for proof, not just attendance.
- ✓ I know where YouTube videos will be posted.
- ✓ I know the weekly contest is part of my score.
- ✓ I know how to write a mistake log.
- ✓ I know how to ask a good question in the group.
- ✓ I understand that copying solutions destroys my own progress.

SECTION 11

Mega Example Lab

A large set of solved examples, mini-patterns, and drills students can review before opening the official sheet.

- Implementation
- conditions
- loops
- arrays

11

Example Lab - salary bonus

Problem

Given salary and years of experience, add bonus if years >= 3.

Approach

- Read salary and years.
- If years >= 3, add 10% bonus.
- Use double if result can be decimal.
- Print final salary.

```
double salary; int years;
cin >> salary >> years;
if (years >= 3) salary *= 1.10;
cout << fixed << setprecision(2) << salary << '\n';
```

Edge checks years=2 | years=3 | salary=0

Example Lab - ticket price

Problem

Read age and print FREE for children under 6, CHILD for under 18, ADULT otherwise.

Approach

- Order conditions from youngest to oldest.
- Use else if chain.
- Check boundaries 5, 6, 17, 18.

```
int age; cin >> age;
if (age < 6) cout << "FREE\n";
else if (age < 18) cout << "CHILD\n";
else cout << "ADULT\n";
```

Edge checks 5 | 6 | 17 | 18

Example Lab - product sign

Problem

Given two integers, print POSITIVE if their product is positive, NEGATIVE if negative, ZERO if zero.

Approach

- If either number is zero, product is zero.
- Otherwise signs decide.
- Can multiply safely if constraints small, or compare signs.

```
long long a,b; cin >> a >> b;
if (a == 0 || b == 0) cout << "ZERO\n";
else if ((a > 0 && b > 0) || (a < 0 && b < 0)) cout << "POSITIVE\n";
else cout << "NEGATIVE\n";
```

Edge checks 0 | both negative | one negative

Example Lab - multiples in range

Problem

Given n and k, count numbers from 1 to n divisible by k.

Approach

- Loop from 1 to n.
- Check i % k == 0.
- Increase counter.
- Later phases may optimize with n/k.

```
int n,k; cin >> n >> k;
int cnt = 0;
for (int i=1;i<=n;i++)
    if (i % k == 0) cnt++;
cout << cnt << '\n';
```

Edge checks k=1 | k>n | n divisible by k

Example Lab - factorial small

Problem

Given n <= 20, print n factorial.

Approach

- Start product at 1.
- Multiply by every number from 1 to n.
- Use long long for small factorials.
- Understand constraints.

```
int n; cin >> n;
long long fact = 1;
for (int i=1;i<=n;i++) fact *= i;
cout << fact << '\n';
```

Edge checks 0 | 1 | 20

Example Lab - array average

Problem

Given n numbers, print their average with two decimals.

Approach

- Use long long sum.
- Read and add all values.
- Convert to double by multiplying by 1.0.
- Use fixed and setprecision.

```
int n; cin >> n;
long long sum = 0;
for (int i=0;i<n;i++){ int x; cin >> x; sum += x; }
double avg = 1.0 * sum / n;
cout << fixed << setprecision(2) << avg << '\n';
```

Edge checks n=1 | sum not divisible | large values

Example Lab - second maximum

Problem

Given n >= 2 distinct integers, print the second largest.

Approach

- Read first two values and initialize mx and second.
- For each new value, update carefully.
- If x > mx, old mx becomes second.
- If x is between mx and second, update second.

```
int n; cin >> n;
int a,b; cin >> a >> b;
int mx = max(a,b), second = min(a,b);
for(int i=2;i<n;i++){
    int x; cin >> x;
    if(x > mx){ second = mx; mx = x; }
    else if(x > second){ second = x; }
}
cout << second << '\n';
```

Edge checks largest first | largest last | n=2

Example Lab - count character

Problem

Given string s and character c, count occurrences of c.

Approach

- Loop over all characters.
- Compare s[i] with c.
- Increase counter on match.
- Print counter.

```
string s; char c;
cin >> s >> c;
int cnt = 0;
for(char ch : s) if(ch == c) cnt++;
cout << cnt << '\n';
```

Edge checks not found | all same | length 1

Example Lab - numeric string sum

Problem

Given a string of digits, print the sum of its digits.

Approach

- Loop over characters.
- Convert digit char to value using c - '0'.
- Add to sum.
- Print sum.

```
string s; cin >> s;
int sum = 0;
for(char c : s) sum += c - '0';
cout << sum << '\n';
```

Edge checks 0 | 999 | 1001

Example Lab - function for max

Problem

Write maxOfThree function and use it in main.

Approach

- Function receives three integers.
- Use max twice or manual comparisons.
- Return the result.
- main prints returned value.

```
int maxOfThree(int a,int b,int c){
    return max(a, max(b,c));
}
int main(){
    int a,b,c; cin >> a >> b >> c;
    cout << maxOfThree(a,b,c) << '\n';
}
```

Edge checks ties | negative | all equal

SECTION 12

Phase 1 Problem Bank

This bank gives students a massive warm-up inventory before the official Codeforces and LeetCode sheets.

- Required
- Practice
- Challenge
- Bonus

12

Input, Output, and Variables

Drill 1

Sum of Two

Read two integers and print their sum.

Drill 2

Swap Output

Read a and b, then print b and a.

Drill 3

Rectangle Area

Read width and height and print area using long long.

Drill 4

Circle Approx

Read radius and print approximate area using pi = 3.14159.

Drill 5

Minutes Converter

Read total minutes and print hours and minutes.

Drill 6

Price After Tax

Read price and tax percentage, print final price.

Conditions

Drill 1

Even/Odd

Classify a number by parity.

Drill 2

Sign Check

Print NEGATIVE, ZERO, or POSITIVE.

Drill 3

In Range

Check whether x is between l and r inclusive.

Drill 4

Max of Three

Print the maximum of three values.

Drill 5

Valid Triangle

Check whether three sides form a valid triangle.

Drill 6

Leap Year Lite

Implement simple leap year rules.

Loops

Drill 1

Count 1 to N

Print numbers from 1 to n.

Drill 2

Sum N Numbers

Read n values and print their sum.

Drill 3

Count Multiples

Count values divisible by k.

Drill 4

Digit Count

Count digits of n.

Drill 5

Reverse Number

Reverse digits of n.

Drill 6

Pattern Square

Print an n by n square of stars.

Math Basics

Drill 1

Last Digit

Print last digit using modulo.

Drill 2

Digit Sum

Print sum of digits.

Drill 3

Ceil Boxes

Calculate number of boxes needed.

Drill 4

Absolute Difference

Print absolute difference between a and b.

Drill 5

Clock Add

Add minutes to current time.

Drill 6

Safe Product

Multiply large values safely.

Arrays

Drill 1

Array Sum

Calculate sum of all elements.

Drill 2

Array Max

Find maximum element.

Drill 3

Array Min

Find minimum element.

Drill 4

Count Greater

Count elements greater than x.

Drill 5

Reverse Array

Print array in reverse order.

Drill 6

First Occurrence

Print first index of target or -1.

Strings

Drill 1

String Length

Print the string length.

Drill 2

First Last

Print first and last character.

Drill 3

Count Vowels

Count vowels in lowercase word.

Drill 4

Reverse Word

Print word reversed.

Drill 5

Palindrome Word

Check if word is palindrome.

Drill 6

Digit String Sum

Sum digits in a numeric string.

Functions

Drill 1

isEven

Write function that returns true if x is even.

Drill 2

maxOfThree

Write function returning max of three numbers.

Drill 3

sumDigits

Write function returning sum of digits.

Drill 4

isPalindrome

Write function checking palindrome string.

Drill 5

countChar

Write function counting target char in string.

Drill 6

arrayMax

Write function returning maximum value in vector.

Mixed Challenge

Drill 1

Password Strength

Classify password based on length, digit, uppercase, lowercase.

Drill 2

Attendance Report

Read n attendance values and count absent/present/late.

Drill 3

Store Receipt

Read item prices and apply discount rules.

Drill 4

Scoreboard

Read scores and print max, min, average.

Drill 5

Robot Moves

Read string of L/R/U/D and calculate final position.

Drill 6

Simple Compression

Count consecutive equal characters in a string.

SECTION 13

Official Phase 1 Sheet Blueprint

The next pages define exactly how the official sheet should be structured and how students should progress through it.

- Required
- Practice
- Challenge
- Bonus

13

Recommended Phase 1 Sheet Distribution

Section	Topics	Problems	Purpose
Required A	I/O, variables, conditions	12	Minimum beginner logic
Required B	loops, math basics	12	Repetition and simulation
Required C	arrays and strings	12	Storing and scanning data
Practice	mixed implementation	15	Speed and confidence
Challenge	tricky edge cases	10	Leaderboard separation
Bonus	harder implementation	5	Top performers

Every Problem Row Should Contain

Column	Meaning	Student Action	Mentor Use
ID	Unique problem number	Track progress	Refer to problem quickly
Topic	Concept being tested	Review related section	Detect weak concepts
Platform	Codeforces/LeetCode/Custom	Open correct link	Separate tracks
Level	Required/Practice/Challenge	Know priority	Score calculation
Deadline	Expected completion date	Avoid last-day rush	Checkpoint review
Status	Not Started/Tried/Solved/Reviewed	Self-manage	Progress tracking
Notes	Mistakes and approach	Explain solution	Review understanding

Every 3 Days - Checkpoint Logic

Checkpoint	Expected Progress	LeetCode Drop	Action
Day 3	Setup + 30% Required	Drop 1 complete	Early warning for inactive students
Day 6	60-70% Required + practice start	Drop 2 complete	Prepare for contest
Day 7	Contest participation	Drop 3 released or reviewed	Submission checkpoint
Day 9	Fixes and mistake log	All drops updated	Recovery tasks if needed
Day 10	Final proof	Streak score locked	Leaderboard + Phase 2 gate

SECTION 14

Ready-to-Execute Pack

This is what should be sent to students when Phase 1 opens.

- files
- links
- forms
- videos

14

What Students Receive on Phase 1 Release Day

- ✓ Phase 1 Mega Recap Guide - this file.
- ✓ Phase 1 Weekly Concept PDF - Week 1 execution file.
- ✓ Codeforces Phase 1 Sheet link with Required/Practice/Challenge/Bonus sections.
- ✓ LeetCode Streak Sheet link with every-three-days drop system.
- ✓ YouTube playlist link for concept videos and solved problems.
- ✓ GitHub repository structure template.
- ✓ Submission form link and exact deadline.
- ✓ Contest date, time, rules, and platform link.
- ✓ Leaderboard explanation and scoring rubric.
- ✓ Question format rules for group support.
- ✓ Mistake log template.
- ✓ Phase 1 final checklist before moving to Phase 2.

Mentor Checklist Before Publishing

- ✓ All Codeforces links are tested and public.
- ✓ YouTube video links are working.
- ✓ Leaderboard sheet formulas are ready.
- ✓ GitHub template is clear.
- ✓ Phase 1 deadline is visible in every channel.
- ✓ Top performer badge rules are defined.
- ✓ LeetCode drop dates are written clearly.
- ✓ Submission form collects all required proof.
- ✓ The first contest is created or scheduled.
- ✓ Students know where to ask questions.
- ✓ Recovery task rules are defined.
- ✓ The next phase gate is explained.

The Phase 1 Standard

What students must understand

- You are not here to collect files. You are here to build the habits of a real problem solver.
- Every problem must leave evidence: code, notes, a test, or a lesson.
- Every three days, your LeetCode streak proves consistency.
- Every Codeforces section proves contest readiness.
- Every submission proves ownership.

The rule

- Watching is not enough.
- Copying is not learning.
- Submitting without proof is not accepted.
- A wrong solution with honest analysis is better than fake completion.
- Phase 1 is the foundation of everything that comes after it.

PHASE 1 IS NOW READY

Students can now start the official sheet with a clear system: concept file, Codeforces sheet, LeetCode drops every three days, YouTube explanations, GitHub proof, submission form, contest, review, and leaderboard.

Think | **Solve** | **Submit** | **Review** | **Improve**

TechMaster Academy